

Interpolación cúbica y sincronización del sistema esclavo de Rössler

Fundamento matemático y su implementación en el receptor

Proyecto: Encriptación con sistema dinámico de Rössler

17 de agosto de 2026

Resumen

El sistema esclavo debe reconstruir la señal caótica que se utilizó para cifrar la imagen. Para lograrlo integra su propio sistema de Rössler forzado por una señal de acoplamiento procedente del maestro. Sin embargo, esa señal no llega como una función continua, sino como una lista finita de muestras transmitidas por MQTT. Este documento explica por qué es necesario reconstruir una función continua a partir de esas muestras, en qué consiste matemáticamente la interpolación por splines cúbicos, y cómo la calidad de esa reconstrucción determina la precisión de la sincronización y, en última instancia, la fidelidad de la imagen descifrada. La descripción se apoya en el programa `Esclavo_TLS_KEYSTREAM.py`.

Índice

1. El problema a resolver	3
1.1. Los dos sistemas	3
1.2. La dificultad	3
2. Interpolación por splines cúbicos	4
2.1. Definición	4
2.2. Naturaleza no local del spline	5
2.3. Precisión	5
2.4. Verificación numérica sobre la señal del experimento	5
2.5. Suavidad y control de paso	6
3. Implementación en el receptor	6
3.1. Construcción del interpolante	6
3.2. Malla de salida	7
4. Dinámica de la sincronización	7
4.1. Ecuación del error	7
4.2. Transitorio y ventana útil	8
4.3. Efecto del error de interpolación	8

5. Del error de sincronización al error en la imagen	9
5.1. Propagación	9
5.2. Escalado y cuantización	9
5.3. Sensibilidad a los parámetros	10
6. Resumen	10

1. El problema a resolver

1.1. Los dos sistemas

El maestro integra el sistema de Rössler autónomo:

$$\begin{aligned}\dot{x}_m &= -y_m - z_m, \\ \dot{y}_m &= x_m + a y_m, \\ \dot{z}_m &= b + z_m (x_m - c),\end{aligned}\tag{1}$$

con $a = 0,2$, $b = 0,2$, $c = 5,7$ y condición inicial $(0,1, 0,1, 0,1)$. La componente x_m resultante es la que se suma a la imagen difundida durante la etapa de confusión.

El esclavo integra un sistema estructuralmente idéntico, pero con un término adicional de acoplamiento en la segunda ecuación:

$$\begin{aligned}\dot{x}_s &= -y_s - z_s, \\ \dot{y}_s &= x_s + a y_s + K (y_m(t) - y_s), \\ \dot{z}_s &= b + z_s (x_s - c),\end{aligned}\tag{2}$$

con $K = 2,0$ y condición inicial $(1,0, 1,0, 1,0)$, distinta de la del maestro. Esta discrepancia deliberada en las condiciones iniciales es lo que permite verificar que la convergencia observada se debe efectivamente al acoplamiento y no a una coincidencia de arranque.

En el código, la ecuación (2) aparece así:

```
1 def rossler_esclavo(t, state, y_master_interp, a, b, c, k):
2     x_s, y_s, z_s = state
3     y_m = y_master_interp(t)
4     dxdt = -y_s - z_s
5     dydt = x_s + a * y_s + k * (y_m - y_s)
6     dzdt = b + z_s * (x_s - c)
7     return [dxdt, dydt, dzdt]
```

Listing 1: Campo vectorial del sistema esclavo

1.2. La dificultad

El término $K (y_m(t) - y_s)$ exige conocer el valor de y_m **en cualquier instante** t . El esclavo, en cambio, solo dispone de un conjunto finito de pares

$$\{(t_i, y_i)\}_{i=0}^{N-1}, \quad y_i = y_m(t_i),\tag{3}$$

recibidos dentro del payload JSON en los campos `y_maestro` y `t_maestro`. En la configuración del experimento hay

$$N = \text{TIEMPO_SINC} + \text{KEYSTREAM} = 6000 + 30000 = 36000\tag{4}$$

muestras, uniformemente espaciadas con paso

$$h = t_{i+1} - t_i = 0,01,\tag{5}$$

cubriendo el intervalo temporal $[0, 360]$.

El problema es que el integrador `solve_ivp` con método RK45 **no evalúa el campo vectorial únicamente en los nodos** t_i . El método de Runge–Kutta de orden 5(4) de Dormand–Prince calcula, dentro de cada paso $[t_n, t_n + \Delta t]$, siete etapas intermedias situadas en

$$t_n + c_j \Delta t, \quad c = \left(0, \frac{1}{5}, \frac{3}{10}, \frac{4}{5}, \frac{8}{9}, 1, 1\right). \quad (6)$$

Los valores $c_2 = 0,2$, $c_3 = 0,3$, $c_4 = 0,8$ y $c_5 \approx 0,889$ caen **estrictamente entre** dos nodos consecutivos. A esto se añade que RK45 emplea paso adaptativo: Δt cambia continuamente y no guarda ninguna relación con $h = 0,01$.

En consecuencia, el integrador solicitará valores de y_m en instantes para los cuales no existe ninguna muestra. Se necesita entonces una función continua $\tilde{y}_m(t)$ que coincida con los datos en los nodos y que proporcione valores razonables en los puntos intermedios. Esa función es el **interpolante**.

2. Interpolación por splines cúbicos

2.1. Definición

Un spline cúbico $S(t)$ sobre la partición $t_0 < t_1 < \dots < t_{N-1}$ es una función definida a trozos: en cada subintervalo $[t_i, t_{i+1}]$ se comporta como un polinomio de tercer grado

$$S_i(t) = \alpha_i + \beta_i (t - t_i) + \gamma_i (t - t_i)^2 + \delta_i (t - t_i)^3, \quad (7)$$

y los distintos trozos se ensamblan de modo que el resultado global sea suave.

Con N nodos hay $n = N - 1$ subintervalos y, por tanto, $4n$ coeficientes desconocidos. Estos quedan determinados por las siguientes condiciones:

Condición	N.º de ecuaciones	Significado
$S_i(t_i) = y_i$ y $S_i(t_{i+1}) = y_{i+1}$	$2n$	El spline pasa exactamente por cada muestra.
$S'_{i-1}(t_i) = S'_i(t_i)$	$n - 1$	Sin cambios bruscos de pendiente en los nodos.
$S''_{i-1}(t_i) = S''_i(t_i)$	$n - 1$	Sin cambios bruscos de curvatura.
Condiciones de frontera	2	Cierran el sistema en los extremos.
Total	$4n$	Sistema determinado.

El resultado es una función de clase C^2 : continua, con primera derivada continua y con segunda derivada continua. Esta propiedad es la que la distingue de alternativas más simples y, como se verá, la que resulta decisiva en este contexto.

La construcción no requiere resolver un sistema denso de $4n$ ecuaciones. Reordenando las condiciones en términos de las segundas derivadas $M_i = S''(t_i)$ se obtiene un sistema **tridiagonal** que, para nodos equiespaciados, adopta la forma

$$M_{i-1} + 4M_i + M_{i+1} = \frac{6}{h^2} (y_{i-1} - 2y_i + y_{i+1}), \quad i = 1, \dots, n-1, \quad (8)$$

resoluble en $\mathcal{O}(N)$ operaciones mediante el algoritmo de Thomas. Para $N = 36000$ esto es prácticamente instantáneo, lo cual justifica que el costo de la interpolación sea despreciable frente al de la integración.

2.2. Naturaleza no local del spline

Conviene notar una consecuencia del sistema (8): los coeficientes de cada trozo dependen de **todas** las muestras, no solo de las vecinas inmediatas. Modificar un único valor y_k altera, en principio, el spline completo, aunque el efecto decae de forma exponencial con la distancia al nodo perturbado (aproximadamente con factor $(2 - \sqrt{3})^{|i-k|} \approx 0,268^{|i-k|}$).

Esto tiene una lectura práctica en el experimento: un error aislado en la transmisión de una muestra de `y_maestro` no queda confinado a un subintervalo, pero tampoco contamina toda la señal. Su influencia se extingue tras unos pocos nodos.

2.3. Precisión

Si la función original y_m es suficientemente derivable, el error del spline cúbico satisface la cota clásica

$$\max_{t \in [t_0, t_{N-1}]} |y_m(t) - S(t)| \leq \frac{5}{384} h^4 \max |y_m^{(4)}(t)|. \quad (9)$$

El punto esencial es la dependencia h^4 : reducir el paso a la mitad divide el error entre dieciséis. Con $h = 0,01$ se tiene $h^4 = 10^{-8}$, un factor extremadamente pequeño.

Para contrastar, la interpolación lineal —unir las muestras con segmentos rectos— tiene una cota

$$\max |y_m(t) - L(t)| \leq \frac{1}{8} h^2 \max |y_m''(t)|, \quad (10)$$

con dependencia h^2 : reducir el paso a la mitad solo divide el error entre cuatro.

2.4. Verificación numérica sobre la señal del experimento

Las cotas anteriores son generales. Aplicadas a la trayectoria concreta del maestro producen los siguientes valores. Se integró el sistema (1) con los parámetros del experimento, se construyeron ambos interpolantes sobre las 36000 muestras y se evaluaron en los puntos medios de cada subintervalo, comparando contra la solución densa del integrador tomada como referencia.

Método	Error máximo	Error RMS	Cota teórica
Spline cúbico	$3,0 \times 10^{-7}$	$6,9 \times 10^{-9}$	$1,4 \times 10^{-6}$
Lineal	$2,9 \times 10^{-4}$	$7,0 \times 10^{-5}$	$2,9 \times 10^{-4}$

Ambos resultados respetan sus cotas respectivas. La diferencia práctica es de casi tres órdenes de magnitud: el spline cúbico es unas 950 veces más preciso que la interpolación lineal sobre esta misma señal y con el mismo espaciado.

La comparación relevante, sin embargo, no es entre ambos métodos sino frente a las tolerancias del integrador. El programa fija `rtol = 1e-5` y `atol = 1e-6`, y la amplitud típica de y_m es del orden de 10, de modo que el error admitido por paso ronda 10^{-4} . Se concluye que:

- Con **spline cúbico**, el error de interpolación ($\sim 10^{-7}$) es unas mil veces menor que la tolerancia del integrador. La interpolación no es el factor limitante.
- Con **interpolación lineal**, el error ($\sim 10^{-4}$) es comparable a la tolerancia. El integrador perseguiría una precisión que la propia señal de entrada no posee.

2.5. Suavidad y control de paso

Hay un segundo motivo, independiente de la precisión, para preferir el spline cúbico. El controlador de paso adaptativo de RK45 estima el error local comparando dos soluciones de órdenes 4 y 5, y ajusta Δt suponiendo que el campo vectorial es suave.

La interpolación lineal produce una función continua pero con derivada discontinua en cada nodo: en cada t_i hay un quiebre. Cuando un paso de integración atraviesa uno de esos quiebres, la estimación de error se dispara artificialmente y el controlador reduce Δt . Con 36000 nodos, el integrador encuentra un quiebre cada 0,01 unidades de tiempo, lo que fuerza pasos pequeños de forma sistemática y encarece la integración sin ganar exactitud.

El spline cúbico, al ser C^2 , no presenta quiebres: el controlador puede utilizar pasos largos en las regiones donde la dinámica lo permite. La suavidad, por tanto, no es un refinamiento estético sino un requisito para que el integrador opere con eficiencia.

3. Implementación en el receptor

3.1. Construcción del interpolante

```

1 y_maestro_interp = interp1d(
2     times,
3     y_maestro,
4     kind='cubic',
5     fill_value="extrapolate"
6 )
7
8 sol_esclavo = solve_ivp(
9     fun=rossler_esclavo,
10    t_span=t_span,
11    y0=X0,
12    t_eval=t_eval,
13    args=(y_maestro_interp,
14          ROSSLER_PARAMS['a'],
15          ROSSLER_PARAMS['b'],
16          ROSSLER_PARAMS['c'],
17          K),
18    rtol=1e-5,
19    atol=1e-6,
20    method='RK45'
21 )

```

Listing 2: Interpolante e integración del sistema esclavo

Los elementos a destacar:

- `kind='cubic'` selecciona el spline cúbico descrito en la sección anterior. La construcción del sistema tridiagonal se realiza una sola vez, al crear el objeto; cada evaluación posterior consiste únicamente en localizar el subintervalo y evaluar el polinomio (7), con costo $\mathcal{O}(\log N)$ por llamada.

- `fill_value=.extrapolate` determina el comportamiento fuera del rango $[t_0, t_{N-1}]$. El integrador puede solicitar, en el último paso, un instante marginalmente superior a t_{N-1} ; sin esta opción la llamada produciría un error y la integración se detendría. Con ella, se prolonga el polinomio del último trozo. Es importante entender que la extrapolación de un polinomio cúbico diverge con rapidez y **no es fiable a distancias apreciables**: aquí resulta admisible únicamente porque el exceso es del orden de una fracción de h .
- El objeto `y_maestro_interp` se pasa a `solve_ivp` como argumento y se invoca en cada evaluación del campo vectorial. Para 36000 pasos con siete etapas cada uno, se realizan del orden de $2,5 \times 10^5$ evaluaciones del spline.

3.2. Malla de salida

El vector `t_eval` se define como

```
1 t_eval = np.linspace(0, iteraciones*H, iteraciones)
```

es decir, sobre la misma malla que el maestro. Esto permite comparar directamente ambas trayectoria a trayectoria, sin necesidad de una segunda interpolación:

```
1 error_x = np.abs(x_maestro - x_esclavo)
```

Conviene subrayar que `t_eval` controla únicamente **dónde se reportan** los resultados, no dónde se calculan. Internamente, RK45 avanza con su propio paso adaptativo y produce los valores solicitados mediante interpolación densa. La necesidad del spline sobre y_m persiste íntegramente.

4. Dinámica de la sincronización

4.1. Ecuación del error

Se define el error de sincronización componente a componente:

$$e_x = x_s - x_m, \quad e_y = y_s - y_m, \quad e_z = z_s - z_m. \quad (11)$$

Restando (1) de (2), y suponiendo por el momento que el esclavo dispone de la señal exacta $y_m(t)$, se obtiene

$$\begin{aligned} \dot{e}_x &= -e_y - e_z, \\ \dot{e}_y &= e_x + a e_y - K e_y = e_x + (a - K) e_y, \\ \dot{e}_z &= z_s (x_s - c) - z_m (x_m - c). \end{aligned} \quad (12)$$

La segunda ecuación revela el mecanismo. El coeficiente que multiplica a e_y pasa de $a = 0,2$ (positivo, expansivo) a

$$a - K = 0,2 - 2,0 = -1,8, \quad (13)$$

decididamente negativo. El acoplamiento convierte una dirección inestable en contractiva: cualquier discrepancia en y tiende a extinguirse. A través de la estructura del sistema, esta contracción arrastra también a e_x y e_z hacia cero.

Formalmente, la condición de sincronización se expresa mediante los **exponentes de Lyapunov condicionales** del sistema de error (12): si todos son negativos, la variedad de sincronización $e = 0$ es asintóticamente estable. El valor $K = 2,0$ se sitúa dentro del rango que garantiza esa condición para los parámetros empleados.

4.2. Transitorio y ventana útil

La convergencia no es instantánea. El error decae aproximadamente de forma exponencial,

$$\|e(t)\| \sim \|e(0)\| e^{-\lambda t}, \quad (14)$$

donde $\lambda > 0$ depende de K y de los parámetros del sistema. Como el esclavo arranca en $(1, 1, 1)$ frente a $(0, 1, 0, 1, 0, 1)$ del maestro, el error inicial es apreciable y se requiere un intervalo de espera antes de que las trayectorias sean utilizables.

Esa espera es precisamente el papel de `TIEMPO_SINC = 6000`, equivalente a 60 unidades de tiempo. El código descarta ese tramo:

```
1 x_sinc = sol_esclavo.y[0][time_sinc:]
2 x_sinc = np.resize(x_sinc, nmax)
```

Solo los 30000 puntos posteriores (`KEYSTREAM`) se emplean para el descifrado, y `np.resize` los repite cíclicamente hasta cubrir los $n_{\text{máx}}$ valores de la imagen. La operación es idéntica en el maestro, lo cual asegura que ambos extremos utilicen exactamente el mismo segmento de señal.

4.3. Efecto del error de interpolación

Cuando el esclavo no dispone de y_m exacta sino de $\tilde{y}_m = S(t)$, la ecuación del error incorpora un término de forzamiento:

$$\dot{e}_y = e_x + (a - K) e_y + K \varepsilon(t), \quad \varepsilon(t) = S(t) - y_m(t). \quad (15)$$

El sistema deja de converger a cero y se estabiliza en torno a un valor residual. Para el modo dominante, la magnitud de ese residuo se estima como

$$\|e\|_\infty \lesssim \frac{K}{|a - K|} \|\varepsilon\|_\infty = \frac{2,0}{1,8} \|\varepsilon\|_\infty \approx 1,11 \|\varepsilon\|_\infty. \quad (16)$$

Sustituyendo los valores medidos:

Interpolación	$\ \varepsilon\ _\infty$	Error residual estimado
Cúbica	$3,0 \times 10^{-7}$	$\sim 3,3 \times 10^{-7}$
Lineal	$2,9 \times 10^{-4}$	$\sim 3,2 \times 10^{-4}$

Esta relación es el vínculo cuantitativo entre ambos temas del documento: **el error de interpolación establece un piso por debajo del cual la sincronización no puede mejorar**, por mucho que se prolongue el tiempo de espera o se afinen las tolerancias del integrador.

5. Del error de sincronización al error en la imagen

5.1. Propagación

El descifrado invierte la etapa de confusión mediante una resta directa:

```
1 def revertir_confusion(vector_cifrado, vector_logistico, x_sinc):
2     difusion = vector_cifrado - vector_logistico - x_sinc
3     return difusion
```

Dado que el maestro construyó el vector cifrado como

$$v_i = d_i + \ell_i + x_{m,i}, \quad (17)$$

la resta en el esclavo produce

$$\tilde{d}_i = v_i - \ell_i - x_{s,i} = d_i + (x_{m,i} - x_{s,i}) = d_i - e_{x,i}. \quad (18)$$

El error de sincronización se traslada **aditivamente y sin amplificación** a cada valor de la imagen difundida. No hay realimentación ni acumulación: cada muestra se ve afectada por su propio $e_{x,i}$.

5.2. Escalado y cuantización

El programa aplica a continuación

```
1 difusion = difusion_x * IMG_SCALE # IMG_SCALE = 100
```

que compensa el factor 0,01 aplicado por el maestro. Este escalado **multiplica también el error** por 100. Finalmente, la reconstrucción cuantiza a enteros de 8 bits:

```
1 img_array = np.round(vector_temp * 255.0).clip(0, 255).astype(np.
    uint8)
```

Un valor de píxel se altera únicamente si el error acumulado supera medio nivel de cuantización, es decir

$$100 \times \|e_x\|_\infty \times 255 > 0,5 \iff \|e_x\|_\infty > 1,96 \times 10^{-5}. \quad (19)$$

Comparando con los residuos estimados en la sección anterior:

- Con interpolación cúbica, $\|e_x\|_\infty \approx 3,3 \times 10^{-7}$ queda unas 60 veces por debajo del umbral. La imagen se recupera sin alteración perceptible, y la distancia de Hamming respecto de la original es prácticamente nula.
- Con interpolación lineal, $\|e_x\|_\infty \approx 3,2 \times 10^{-4}$ excede el umbral en más de un orden de magnitud. Aparecen desviaciones de varios niveles de intensidad, detectables en la distancia de Hamming y en el diagrama de dispersión píxel a píxel.

Este cálculo explica por qué la elección del método de interpolación —una decisión aparentemente técnica y menor— tiene consecuencias directas y medibles sobre el resultado del experimento. El proyecto conserva variantes sin interpolación (`Esclavo_TLS_KEYSTREAM NoInterp.py`) precisamente para poder contrastar este efecto.

5.3. Sensibilidad a los parámetros

Los barridos de Hamming implementados en el receptor (`experimento_hamming_vs_a`, `_vs_b`, `_vs_c`) reejecutan la sincronización completa modificando un parámetro de Rössler cada vez. El resultado ilustra un comportamiento distinto del anterior:

Un error de interpolación se propaga de forma **aditiva y acotada**, como se acaba de mostrar. Un error en los parámetros, en cambio, altera la propia dinámica del sistema esclavo: las trayectorias divergen exponencialmente y x_s deja de guardar relación con x_m . La distancia de Hamming pasa entonces de valores próximos a cero a valores cercanos al 50 % de los bits, que es lo esperado al comparar dos secuencias sin correlación.

Ambos mecanismos deben distinguirse al interpretar los resultados: el primero es un error numérico controlable mediante la elección del interpolante; el segundo es la sensibilidad a las condiciones que constituye el fundamento de seguridad del esquema.

6. Resumen

1. El esclavo recibe la señal de acoplamiento y_m como 36000 muestras discretas espaciadas $h = 0,01$.
2. El integrador RK45 requiere evaluar y_m en instantes intermedios arbitrarios, impuestos por sus etapas internas y por el paso adaptativo.
3. El spline cúbico reconstruye una función C^2 que pasa por todas las muestras, resolviendo un sistema tridiagonal en $\mathcal{O}(N)$ operaciones.
4. Su error decae como h^4 : sobre la señal del experimento resulta de $3,0 \times 10^{-7}$, frente a $2,9 \times 10^{-4}$ de la interpolación lineal.
5. La continuidad de las derivadas evita que el controlador de paso adaptativo reduzca Δt innecesariamente en cada nodo.
6. El acoplamiento con $K = 2,0$ convierte el coeficiente a en $a - K = -1,8$, haciendo contractiva la dinámica del error y forzando la sincronización.
7. El error de interpolación actúa como forzamiento residual y fija un piso de aproximadamente $1,11 \|\varepsilon\|_\infty$ para el error de sincronización.
8. Ese error se traslada aditivamente a la imagen; tras el escalado por 100 y la cuantización a 8 bits, la interpolación cúbica queda holgadamente por debajo del umbral de alteración de un píxel, mientras que la lineal lo supera.